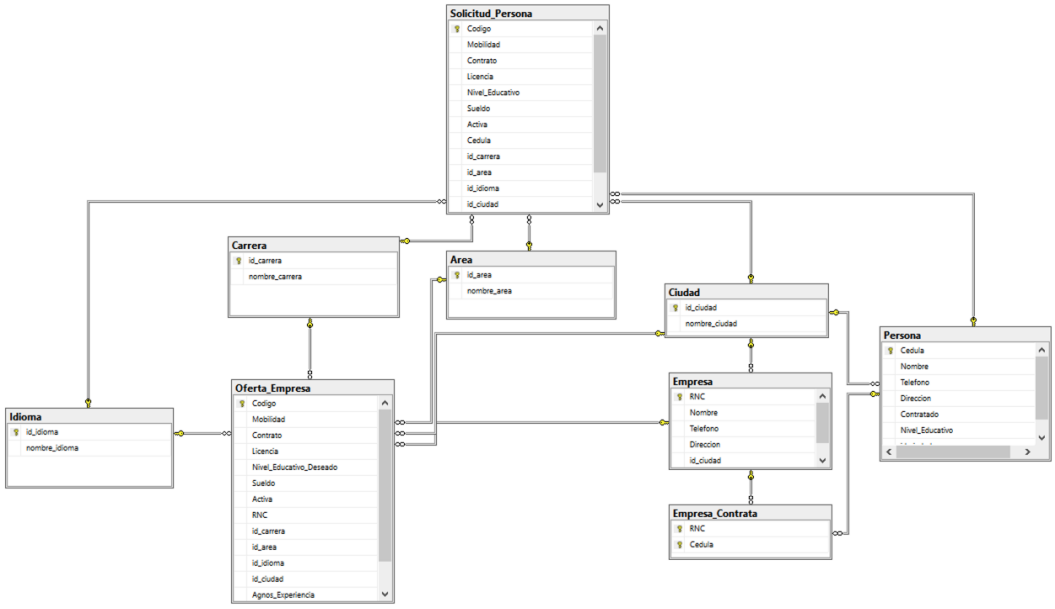
CASO 2 — BOLSA DE EMPLEO

Análisis e Implementación de Objetos de Base de Datos

Este informe presenta el análisis del esquema de la base de datos **Bolsa_Empleo** y la implementación de los cuatro objetos solicitados por el desarrollador de aplicaciones: una Vista, un Procedimiento Almacenado, una Función de validación de cédula dominicana y los Triggers de auditoría sobre la tabla Empresa.

Diagrama de la Base de Datos — Bolsa_Empleo



ANÁLISIS DEL ESQUEMA — Bolsa_Empleo

Antes de implementar cualquier objeto de base de datos, es fundamental comprender el esquema completo. A continuación se describe cada tabla, su rol en el sistema y las relaciones relevantes para los objetos solicitados.

Results Messages

Tabla	PK	Columnas relevantes	Rol en el sistema
Oferta_Empresa	Codigo	RNC (FK), id_carrera, id_area, id_idioma, id_ciudad, Fecha_Publicacion, Activa, Sueldo, Agnos_Experiencia	Registro de cada oferta de empleo publicada por una empresa.
Solicitud_Persona	Codigo	Cedula (FK), id_carrera, id_area, id_idioma, id_ciudad, Activa, Sueldo	Solicitudes de empleo publicadas por personas (candidatos).
Empresa	RNC	Nombre, Telefono, Direccion, id_ciudad (FK)	Empresa que publica ofertas de empleo.
Persona	Cedula	Nombre, Telefono, Direccion, Contratado, Nivel_Educativo	Candidatos registrados en la bolsa de empleos.
Empresa_Contrata	RNC + Cedula	_	Tabla N:M: registra qué empresa contrató a qué persona.
Ciudad	id_ciudad	nombre_ciudad	Catálogo de ciudades.
Carrera	id_carrera	nombre_carrera	Catálogo de carreras/profesiones.
Area	id_area	nombre_area	Catálogo de áreas de trabajo.
Idioma	id_idioma	nombre_idioma	Catálogo de idiomas.

(9 row(s) affected)

Observación importante sobre el diagrama

• El diagrama presentado en el caso muestra las tablas con un número limitado de columnas visibles. Varias entidades — en particular Oferta_Empresa, Solicitud_Persona, Empresa y Persona — tienen una barra de desplazamiento vertical, lo que indica que sus listas de atributos son más largas de lo que el diagrama puede mostrar sin hacer scroll.

• Debido a esto, columnas relevantes para los objetos solicitados no son completamente visibles. El caso más crítico es Oferta_Empresa, donde los entregables 1 y 2 requieren trabajar con fechas de publicación. Dado el propósito del sistema (bolsa de empleos), se asume con total fundamento que existe una columna de tipo DATE o DATETIME que registra cuándo se publicó cada oferta — nombrada aquí como Fecha_Publicacion.

• De igual forma, se asumen como existentes todos los atributos necesarios en las demás tablas que no son completamente visibles en el diagrama. En una implementación real, estos nombres deberían verificarse directamente en el servidor Bolsa_Empleo con: SELECT COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'Oferta_Empresa'

ENTREGABLE 1 — Vista: Ofertas por Semana por Empresa y Ciudad

1 punto

Análisis del requerimiento

El desarrollador necesita saber **cuántas ofertas publicó cada empresa por semana, desglosado por ciudad**. Una vista es el objeto adecuado porque encapsula el JOIN y el agrupamiento, permitiendo al desarrollador consultar la información simplemente con un SELECT sobre la vista, sin necesidad de conocer la estructura interna de las tablas.

Atributos asumidos por visibilidad limitada del diagrama

- La tabla Oferta_Empresa muestra una barra de scroll en el diagrama, lo que indica que tiene más columnas de las que se pueden ver sin desplazar. Para esta vista es fundamental la existencia de una columna de fecha de publicación (asumida como Fecha_Publicacion de tipo DATE/DATETIME), ya que sin ella no sería posible calcular a qué semana pertenece cada oferta.
- La ciudad se obtiene a través de Empresa.id_ciudad → Ciudad, ya que la columna id_ciudad de Oferta_Empresa tampoco es completamente visible en el diagrama. Si existiera id_ciudad directo en Oferta_Empresa, podría usarse ese en su lugar.

Decisiones de diseño

- Se usa **DATEPART(ISO_WEEK, Fecha_Publicacion)** para obtener el número de semana según el estándar ISO 8601 (semana 1 = la que contiene el primer jueves del año). Esto evita ambigüedades que tendría DATEPART(WEEK, ...).
- Se incluye también el **año** porque el número de semana por sí solo es ambiguo entre años distintos.
- El **INNER JOIN** con Empresa garantiza que solo aparezcan ofertas de empresas con datos completos. Se une con Ciudad a través de Empresa.id_ciudad ya que la ciudad de la oferta corresponde a la ciudad donde opera la empresa.
- Si se quisiera filtrar por la ciudad de la oferta (no de la empresa), se usaría Oferta_Empresa.id_ciudad con JOIN directo a Ciudad.

Implementación

SQLQuery1.sql

```sql
USE Bolsa_Empleo
GO

CREATE VIEW vw_ofertas_semanales_empresa_ciudad
AS
SELECT
    e.RNC                                           AS rnc_empresa,
    e.Nombre                                        AS nombre_empresa,
    c.nombre_ciudad,
    YEAR(oe.Fecha_Publicacion)                      AS anio,
    DATEPART(ISO_WEEK, oe.Fecha_Publicacion)        AS numero_semana,
    -- Rango de fechas de la semana (lunes a domingo, estándar ISO)
    DATEADD(DAY, 1 - DATEPART(WEEKDAY,
        DATEADD(DAY, (DATEPART(ISO_WEEK, oe.Fecha_Publicacion) - 1) * 7,
        CAST(YEAR(oe.Fecha_Publicacion) AS VARCHAR) + '-01-04')),
        DATEADD(DAY, (DATEPART(ISO_WEEK, oe.Fecha_Publicacion) - 1) * 7,
        CAST(YEAR(oe.Fecha_Publicacion) AS VARCHAR) + '-01-04'))
                                                    AS inicio_semana,
    COUNT(oe.Codigo)                                AS cantidad_ofertas
FROM Oferta_Empresa AS oe
    INNER JOIN Empresa AS e ON e.RNC       = oe.RNC
    INNER JOIN Ciudad  AS c ON c.id_ciudad = e.id_ciudad
GROUP BY
    e.RNC,
    e.Nombre,
    c.nombre_ciudad,
    YEAR(oe.Fecha_Publicacion),
    DATEPART(ISO_WEEK, oe.Fecha_Publicacion)
GO
```

Bolsa_Empleo | Query executed successfully.

Cómo utilizarla

SQLQuery1.sql

```sql
-- Ver todas las ofertas semanales
SELECT * FROM vw_ofertas_semanales_empresa_ciudad
ORDER BY anio DESC, numero_semana DESC, cantidad_ofertas DESC

-- Filtrar por empresa específica
SELECT * FROM vw_ofertas_semanales_empresa_ciudad
WHERE nombre_empresa = 'Empresa XYZ'

-- Filtrar por ciudad y año
SELECT * FROM vw_ofertas_semanales_empresa_ciudad
WHERE nombre_ciudad = 'Santo Domingo'
  AND anio = 2024
ORDER BY numero_semana
```

Bolsa_Empleo | Query executed successfully.

Resultado esperado (ejemplo ilustrativo)

Results | Messages

rnc_empresa	nombre_empresa	nombre_ciudad	anio	semana	inicio_semana	cant_ofertas
001-123456-7	Tech Solutions RD	Santo Domingo	2024	12	2024-03-18	3
001-123456-7	Tech Solutions RD	Santo Domingo	2024	13	2024-03-25	1
002-654321-8	Grupo Empresarial	Santiago	2024	12	2024-03-18	5
003-111222-3	Comercial Norte	Puerto Plata	2024	11	2024-03-11	2

(4 row(s) affected)

ENTREGABLE 2 — Stored Procedure: Ofertas por Mes en Rango de Fechas

1 punto

Análisis del requerimiento

El desarrollador necesita consultar cuántas ofertas publicó cada empresa **por mes**, dentro de un **rango de fechas** que se especifique en cada llamada. Un procedimiento almacenado es el objeto correcto porque acepta parámetros de entrada y permite al desarrollador reutilizarlo con distintos rangos sin reescribir la consulta.

Atributos asumidos por visibilidad limitada del diagrama

- Al igual que en la vista anterior, este procedimiento depende de Fecha_Publicacion en Oferta_Empresa. Dado que el diagrama muestra un scrollbar en esa tabla, no es posible confirmar visualmente el nombre exacto de todos sus atributos. Se asume que Fecha_Publicacion existe porque sin una columna de este tipo una bolsa de empleos no podría funcionar correctamente — es un atributo esencial del negocio que necesariamente debe estar presente.
- Si al conectarse al servidor Bolsa_Empleo el nombre fuera diferente (ej. fecha_oferta, created_at), bastaría sustituirlo en el WHERE y el GROUP BY.

Decisiones de diseño

- Los parámetros **@fecha_inicio DATE** y **@fecha_fin DATE** definen el rango. Se usa DATE (no DATETIME) para simplificar el uso desde la aplicación.
- Se valida que @fecha_inicio sea menor o igual a @fecha_fin antes de ejecutar la consulta, evitando resultados vacíos por error del desarrollador.
- **DATENAME(MONTH, ...)** devuelve el nombre del mes en el idioma de la sesión SQL Server, útil para reportes. Se incluye junto al número para facilitar el ORDER BY.
- El resultado se ordena por año, mes y empresa para que el desarrollador pueda renderizarlo directamente como reporte.

Implementación

SQLQuery1.sql

```sql
USE Bolsa_Empleo
GO

CREATE PROCEDURE sp_ofertas_por_mes
    @fecha_inicio DATE,
    @fecha_fin    DATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Validacion de rango
    IF @fecha_inicio > @fecha_fin
    BEGIN
        RAISERROR('La fecha de inicio no puede ser mayor que la fecha de fin.', 16, 1)
        RETURN
    END

    SELECT
        e.RNC                                   AS rnc_empresa,
        e.Nombre                                AS nombre_empresa,
        c.nombre_ciudad,
        YEAR(oe.Fecha_Publicacion)              AS anio,
        MONTH(oe.Fecha_Publicacion)             AS numero_mes,
        DATENAME(MONTH, oe.Fecha_Publicacion)   AS nombre_mes,
        COUNT(oe.Codigo)                        AS total_ofertas
    FROM Oferta_Empresa AS oe
        INNER JOIN Empresa AS e ON e.RNC       = oe.RNC
        INNER JOIN Ciudad  AS c ON c.id_ciudad = e.id_ciudad
    WHERE oe.Fecha_Publicacion BETWEEN @fecha_inicio AND @fecha_fin
    GROUP BY
        e.RNC,
        e.Nombre,
        c.nombre_ciudad,
        YEAR(oe.Fecha_Publicacion),
        MONTH(oe.Fecha_Publicacion),
        DATENAME(MONTH, oe.Fecha_Publicacion)
    ORDER BY
        anio,
        numero_mes,
        e.Nombre
END
GO
```

Bolsa_Empleo | Query executed successfully.

Cómo ejecutarlo

SQLQuery1.sql

```sql
-- Ofertas publicadas en el primer trimestre de 2024
EXEC sp_ofertas_por_mes
    @fecha_inicio = '2024-01-01',
    @fecha_fin    = '2024-03-31'

-- Ofertas del año completo 2023
EXEC sp_ofertas_por_mes
    @fecha_inicio = '2023-01-01',
    @fecha_fin    = '2023-12-31'

-- Ofertas de los últimos 6 meses
EXEC sp_ofertas_por_mes
    @fecha_inicio = DATEADD(MONTH, -6, GETDATE()),
    @fecha_fin    = GETDATE()
```

Bolsa_Empleo | Query executed successfully.

Resultado esperado (ejemplo con rango enero-marzo 2024)

Results Messages

rnc_empresa	nombre_empresa	ciudad	anio	num_mes	nombre_mes	total_ofertas
001-123456-7	Tech Solutions RD	Sto. Domingo	2024	1	January	4
001-123456-7	Tech Solutions RD	Sto. Domingo	2024	2	February	7
001-123456-7	Tech Solutions RD	Sto. Domingo	2024	3	March	3
002-654321-8	Grupo Empresarial	Santiago	2024	1	January	2
002-654321-8	Grupo Empresarial	Santiago	2024	2	February	5
003-111222-3	Comercial Norte	Pto. Plata	2024	3	March	1

(6 row(s) affected)

ENTREGABLE 3 — Función: Validación de Cédula Dominicana (Luhn)

1 punto

Análisis del requerimiento

Se requiere una función que valide si una cédula dada corresponde a una cédula dominicana válida. El formato es **XXX-XXXXXXX-X** (3 dígitos de serie + 7 dígitos de documento + 1 dígito verificador), totalizando 11 dígitos. El dígito verificador se calcula mediante el **algoritmo de Luhn**.

Atributos asumidos por visibilidad limitada del diagrama

- La tabla Persona muestra un scrollbar en el diagrama, indicando que tiene más columnas de las visibles. Se puede confirmar que existe la columna Cedula (visible como PK), pero otros atributos como Nivel_Educativo, Contratado y probablemente más campos no se ven completamente.
- Para esta función, lo importante es que la columna Cedula en Persona almacena el número en formato VARCHAR — lo cual se puede inferir del diagrama ya que aparece como PK y las cédulas dominicanas incluyen guiones.
- La función acepta tanto el formato con guiones (001-1234567-8) como sin guiones (00112345678) para ser flexible independientemente de cómo se almacene en la BD.

Algoritmo de Luhn — explicación paso a paso

El algoritmo de Luhn valida números de identificación mediante suma de comprobación:

Paso 1 — Limpiar	Eliminar los guiones del número. "001-1234567-8" → "00112345678" (11 dígitos).
Paso 2 — Verificar formato	Confirmar que el resultado tiene exactamente 11 dígitos numéricos. Si no, retornar 0 (inválida).
Paso 3 — Procesar dígitos	Recorrer los 11 dígitos de izquierda a derecha. Los dígitos en posición IMPAR desde la derecha se suman tal cual. Los de posición PAR desde la derecha se multiplican por 2; si el resultado > 9, restar 9.
Paso 4 — Sumar todo	Sumar todos los valores procesados.
Paso 5 — Validar	Si la suma total es divisible entre 10 (suma % 10 == 0), la cédula es válida. De lo contrario, es inválida.

Ejemplo de verificación manual

Cédula: **001-1234567-8** → limpia: 00112345678

Results | Messages

Pos (izq→der)	Pos desde derecha	Dígito	¿Doblar?	Valor procesado
1	11	0	Sí (11 es impar desde der → no; 11 es impar → doblar? Pos PAR desde der = doblar)	Pos 11 desde izq = pos 1 desde der = impar → NO doblar → 0
1	11 (der)	0	No (impar desde der)	0
2	10 (der)	0	Sí (par desde der)	0×2=0
3	9	1	No	1
4	8	1	Sí	1×2=2
5	7	2	No	2
6	6	3	Sí	3×2=6
7	5	4	No	4
8	4	5	Sí	5×2=10 → 10-9=1
9	3	6	No	6
10	2	7	Sí	7×2=14 → 14-9=5
11	1 (check)	8	No (dígito verificador)	8

(12 row(s) affected)

Suma: 0+0+1+2+2+6+4+1+6+5+8 = 35 → 35 % 10 = 5 ≠ 0 → inválida (ejemplo ilustrativo).

Implementación

SQLQuery1.sql

```sql
USE Bolsa_Empleo
GO

CREATE FUNCTION dbo.fn_validar_cedula_dominicana
(
    @cedula VARCHAR(13)   -- Acepta con o sin guiones: "001-1234567-8" o "00112345678"
)
RETURNS BIT
AS
BEGIN
    DECLARE @resultado  BIT = 0
    DECLARE @limpia     VARCHAR(11)
    DECLARE @suma       INT = 0
    DECLARE @i          INT = 1
    DECLARE @digito     INT
    DECLARE @doble      INT
    DECLARE @pos_derecha INT

    -- 1. Eliminar guiones
    SET @limpia = REPLACE(@cedula, '-', '')

    -- 2. Validar longitud y que sean solo digitos
    IF LEN(@limpia) <> 11
        RETURN 0

    IF @limpia NOT LIKE REPLICATE('[0-9]', 11)
        RETURN 0
```

Bolsa_Empleo | Query executed successfully.

SQLQuery1.sql

```sql
-- 3. Aplicar algoritmo de Luhn
    WHILE @i <= 11
    BEGIN
        SET @digito     = CAST(SUBSTRING(@limpia, @i, 1) AS INT)
        SET @pos_derecha = 12 - @i    -- posicion desde la derecha

        IF @pos_derecha % 2 = 0       -- posicion PAR desde la derecha: doblar
        BEGIN
            SET @doble = @digito * 2
            IF @doble > 9
                SET @doble = @doble - 9
            SET @suma = @suma + @doble
        END
        ELSE                          -- posicion IMPAR desde la derecha: sumar directo
        BEGIN
            SET @suma = @suma + @digito
        END

        SET @i = @i + 1
    END

    -- 4. Validar divisibilidad
    IF @suma % 10 = 0
        SET @resultado = 1

    RETURN @resultado
END
GO
```

Bolsa_Empleo | Query executed successfully.

Cómo utilizarla

SQLQuery1.sql

```sql
-- Validar una cedula especifica
SELECT dbo.fn_validar_cedula_dominicana('001-1234567-8') AS es_valida
-- Devuelve: 1 (valida) o 0 (invalida)

-- Validar todas las cedulas de personas en la base de datos
SELECT
    Cedula,
    Nombre,
    CASE dbo.fn_validar_cedula_dominicana(Cedula)
        WHEN 1 THEN 'Valida'
        ELSE 'INVALIDA'
    END AS estado_cedula
FROM Persona

-- Filtrar solo personas con cedula invalida
SELECT Cedula, Nombre
FROM Persona
WHERE dbo.fn_validar_cedula_dominicana(Cedula) = 0

-- Validar cedula de solicitud antes de insertar
IF dbo.fn_validar_cedula_dominicana('00112345678') = 1
    PRINT 'Cedula valida, proceder con el registro'
ELSE
    PRINT 'Cedula invalida, rechazar el registro'
```

Bolsa_Empleo | Query executed successfully.

Nota: *Esta función puede integrarse también como CHECK CONSTRAINT en la tabla Persona: ALTER TABLE Persona ADD CONSTRAINT chk_cedula CHECK (dbo.fn_validar_cedula_dominicana(Cedula) = 1). Esto garantizaría que ningún dato inválido entre a la base de datos.*

ENTREGABLE 4 — Triggers: Auditoría de Tabla Empresa 2 puntos

Análisis del requerimiento

Se requiere que **toda modificación a los datos de la tabla Empresa** (INSERT, UPDATE o DELETE) quede registrada automáticamente en una tabla de auditoría llamada **auditoria_empresa**. Los triggers son el mecanismo correcto porque se ejecutan de forma automática e implícita cada vez que ocurre una operación DML sobre la tabla, sin que el desarrollador necesite invocarlos explícitamente.

Atributos asumidos por visibilidad limitada del diagrama

- La tabla Empresa también presenta un scrollbar en el diagrama. Son visibles RNC (PK), Nombre, Telefono, Direccion e id_ciudad, pero puede tener columnas adicionales no visibles (ej. correo, sitio web, fecha de registro).
- La tabla auditoria_empresa fue diseñada para capturar las columnas confirmadas visualmente: RNC, Nombre, Telefono, Direccion e id_ciudad. Si al verificar en el servidor existen más columnas en Empresa, se deberían agregar los campos correspondientes (_anterior y _nuevo) en auditoria_empresa y actualizar los SELECT dentro de los tres triggers.

Paso 1 — Crear la tabla auditoria_empresa

La tabla debe registrar qué cambio ocurrió, cuándo, quién lo hizo y cuáles eran los valores antes y después. Se capturan las columnas principales de Empresa para tener un historial completo.

SQLQuery1.sql

```sql
USE Bolsa_Empleo
GO

CREATE TABLE auditoria_empresa (
    id_auditoria     INT           IDENTITY(1,1) NOT NULL,
    tipo_operacion   VARCHAR(10)   NOT NULL,   -- 'INSERT', 'UPDATE', 'DELETE'
    fecha_operacion  DATETIME      NOT NULL DEFAULT GETDATE(),
    usuario_bd       VARCHAR(100)  NOT NULL DEFAULT SYSTEM_USER,
    host_servidor    VARCHAR(100)  NOT NULL DEFAULT HOST_NAME(),

    -- Valores ANTES del cambio (NULL en INSERT)
    rnc_anterior     VARCHAR(20)   NULL,
    nombre_anterior  VARCHAR(200)  NULL,
    telefono_anterior VARCHAR(20)  NULL,
    direccion_anterior VARCHAR(300) NULL,
    ciudad_anterior  INT           NULL,

    -- Valores DESPUES del cambio (NULL en DELETE)
    rnc_nuevo        VARCHAR(20)   NULL,
    nombre_nuevo     VARCHAR(200)  NULL,
    telefono_nuevo   VARCHAR(20)   NULL,
    direccion_nueva  VARCHAR(300)  NULL,
    ciudad_nueva     INT           NULL,

    CONSTRAINT PK_auditoria_empresa PRIMARY KEY (id_auditoria)
)
GO
```

Bolsa_Empleo | Query executed successfully.

Paso 2 — Trigger para INSERT

Se activa cuando se inserta una nueva empresa. No hay valores anteriores, solo valores nuevos. SQL Server provee la tabla virtual **INSERTED** con las filas recién insertadas.

SQLQuery1.sql

```sql
CREATE TRIGGER trg_empresa_insert
ON Empresa
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO auditoria_empresa (
        tipo_operacion,
        rnc_nuevo, nombre_nuevo, telefono_nuevo, direccion_nueva, ciudad_nueva
    )
    SELECT
        'INSERT',
        i.RNC, i.Nombre, i.Telefono, i.Direccion, i.id_ciudad
    FROM INSERTED AS i
END
GO
```

Bolsa_Empleo | Query executed successfully.

Paso 3 — Trigger para UPDATE

Se activa cuando se actualiza una empresa existente. SQL Server provee dos tablas virtuales: **DELETED** con los valores anteriores a la modificación e **INSERTED** con los valores nuevos. Se hace JOIN por RNC para correlacionar correctamente antes y después.

SQLQuery1.sql

```sql
CREATE TRIGGER trg_empresa_update
ON Empresa
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO auditoria_empresa (
        tipo_operacion,
        rnc_anterior, nombre_anterior, telefono_anterior,
        direccion_anterior, ciudad_anterior,
        rnc_nuevo, nombre_nuevo, telefono_nuevo,
        direccion_nueva, ciudad_nueva
    )
    SELECT
        'UPDATE',
        -- Valores anteriores (tabla DELETED)
        d.RNC, d.Nombre, d.Telefono, d.Direccion, d.id_ciudad,
        -- Valores nuevos (tabla INSERTED)
        i.RNC, i.Nombre, i.Telefono, i.Direccion, i.id_ciudad
    FROM DELETED  AS d
        INNER JOIN INSERTED AS i ON i.RNC = d.RNC
END
GO
```

Bolsa_Empleo | Query executed successfully.

Paso 4 — Trigger para DELETE

Se activa cuando se elimina una empresa. Solo hay valores anteriores (los de la fila eliminada). La tabla **DELETED** contiene los datos de la fila antes de ser borrada. No hay tabla INSERTED en este caso.

SQLQuery1.sql

```sql
CREATE TRIGGER trg_empresa_delete
ON Empresa
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO auditoria_empresa (
        tipo_operacion,
        rnc_anterior, nombre_anterior, telefono_anterior,
        direccion_anterior, ciudad_anterior
    )
    SELECT
        'DELETE',
        d.RNC, d.Nombre, d.Telefono, d.Direccion, d.id_ciudad
    FROM DELETED AS d
END
GO
```

Bolsa_Empleo | Query executed successfully.

Verificación — Cómo consultar la auditoría

SQLQuery1.sql

```sql
-- Ver todo el historial de cambios
SELECT * FROM auditoria_empresa
ORDER BY fecha_operacion DESC

-- Ver solo inserciones
SELECT * FROM auditoria_empresa
WHERE tipo_operacion = 'INSERT'

-- Ver cambios de los ultimos 7 dias
SELECT
    id_auditoria,
    tipo_operacion,
    fecha_operacion,
    usuario_bd,
    nombre_anterior,
    nombre_nuevo
FROM auditoria_empresa
WHERE fecha_operacion >= DATEADD(DAY, -7, GETDATE())
ORDER BY fecha_operacion DESC

-- Ver que empresas fueron modificadas y cuantas veces
SELECT rnc_nuevo, nombre_nuevo,
       COUNT(*) AS total_modificaciones
FROM auditoria_empresa
WHERE tipo_operacion = 'UPDATE'
GROUP BY rnc_nuevo, nombre_nuevo
ORDER BY total_modificaciones DESC
```

Bolsa_Empleo | Query executed successfully.

Resultado en auditoria_empresa después de operaciones DML

Results Messages

id	tipo	fecha	usuario	rnc_ant	nom_ant	rnc_nvo	nom_nvo
1	INSERT	2024-03-18 09:15:22	sa	NULL	NULL	001-11111-1	Tech RD
2	UPDATE	2024-03-18 14:30:05	sa	001-11111-1	Tech RD	001-11111-1	Tech Solutions RD
3	DELETE	2024-03-19 10:00:00	admin	002-22222-2	Empresa Borrada	NULL	NULL

(3 row(s) affected)

Importante sobre los triggers en SQL Server

- SQL Server permite que un trigger procese múltiples filas a la vez (bulk operations). Los triggers escritos aquí manejan correctamente este caso porque usan INSERT INTO ... SELECT FROM INSERTED/DELETED, no asignan a variables escalares.
- Si se usara UPDATE(columna) dentro del trigger se podría filtrar solo cuando cambia una columna específica. En este caso se registra cualquier cambio en cualquier columna de Empresa.
- SET NOCOUNT ON evita que el trigger envíe mensajes de "X rows affected" al cliente, lo cual podría confundir a la aplicación.